SYSTEM HARDENING

Provisioning and Hardening Two Digital Ocean Droplets with DISA STIGs Using Ansible Automation Tool

KHRISTOPHER WALDEN

COURSE: PROJECT & PORTFOLIO 7

TABLE OF CONTENTS

Scope of Work

PROJECT SCOPE OVERVIEW

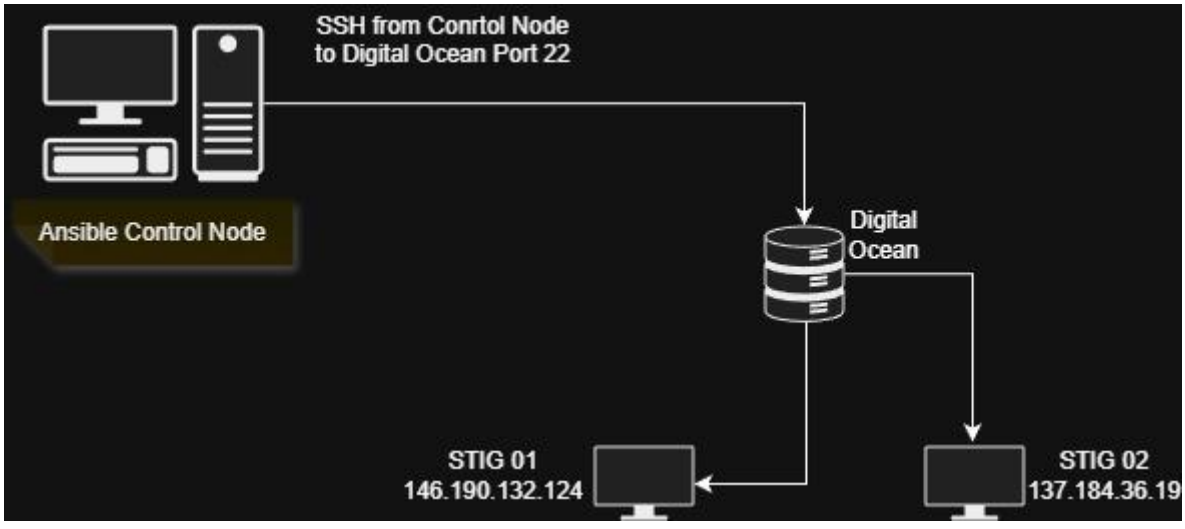
The purpose of this project is to provision and harden two Ubuntu Linux virtual machines hosted in the DigitalOcean cloud using Ansible automation and DISA Security Technical Implementation Guide (STIG) principles. The project focuses on implementing automated system hardening to ensure consistent, repeatable, and secure configurations across multiple cloud-based systems.

The scope of the project includes the creation of two identical DigitalOcean droplets running Ubuntu Linux and the configuration of an Ansible control node used to remotely manage and enforce security policies on the target systems. Secure communication between the control node and the target droplets is established using SSH key-based authentication. Ansible is utilized to automate the application of security controls, eliminating manual configuration and reducing the risk of configuration drift.

As part of the hardening process, a total of twenty DISA STIG-aligned security controls are implemented and categorized by risk severity, including five Category I (High), five Category II (Medium), and ten Category III (Low) controls. These controls address areas such as SSH hardening, firewall enforcement, auditing, time synchronization, kernel network protections, file permission enforcement, and system update management.

The success of the project is measured by the successful execution of Ansible playbooks, validation of applied configurations using command-line verification, and confirmation that both target systems reach an identical hardened security posture. The project is designed to be achievable using open-source tools, realistic within the assigned milestone timeline, and aligned with industry-recognized security compliance practices.

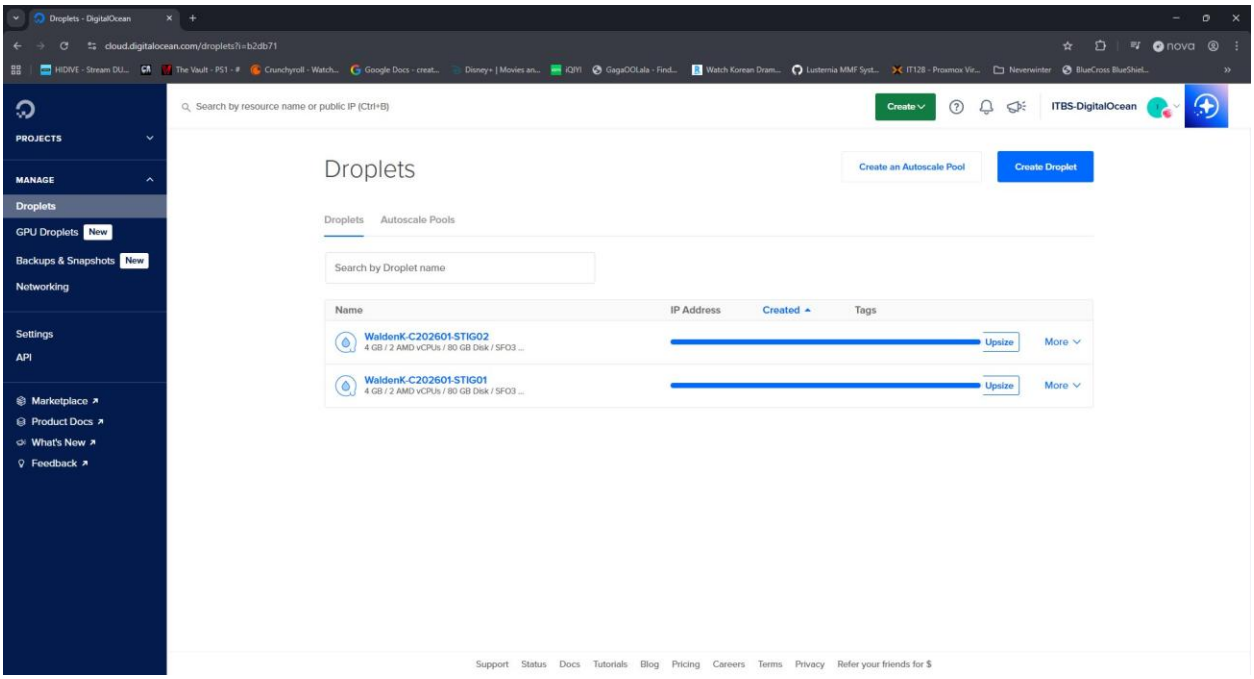
NETWORK TOPOLOGY DIAGRAM WITH IP ADDRESSES



CLOUD AUTOMATION PROCEDURAL INSTRUCTIONS

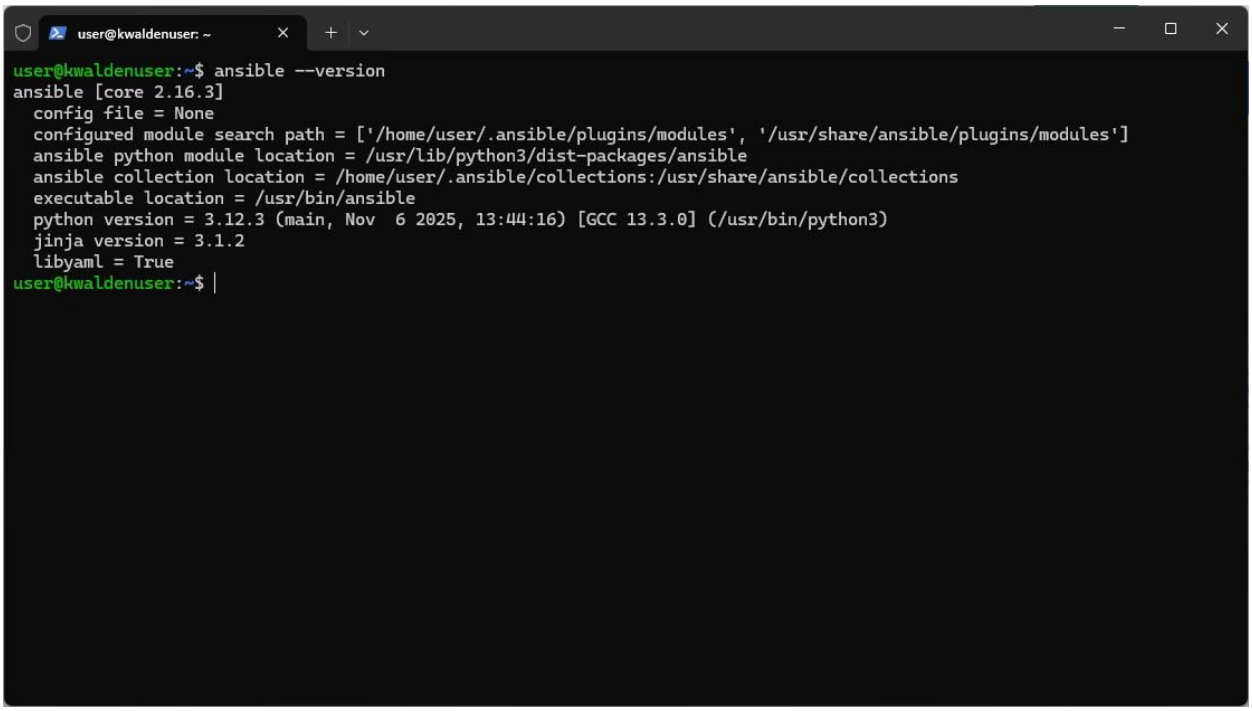
Step 1: Provision Ubuntu Droplets in DigitalOcean

1. Log in to the DigitalOcean control panel.
2. Create two new droplets using identical configurations:
    - Operating System: Ubuntu Linux (LTS version)
    - Droplet size: Basic plan
    - Region: Same region for both droplets
3. Assign the following hostnames:
    - stig-target-01
    - stig-target-02
4. Enable SSH key authentication during droplet creation.
5. Complete the provisioning process and verify both droplets are running.
6. Record the public IP addresses assigned to each droplet for later use.



Step 2: Prepare the Ansible Control Node

1. Provision a separate Ubuntu Linux droplet to act as the Ansible control node.
2. Log in to the control node via SSH.
3. Update and upgrade the operating system packages.
4. Install Ansible and required dependencies.
5. Verify the installation by checking the Ansible version.



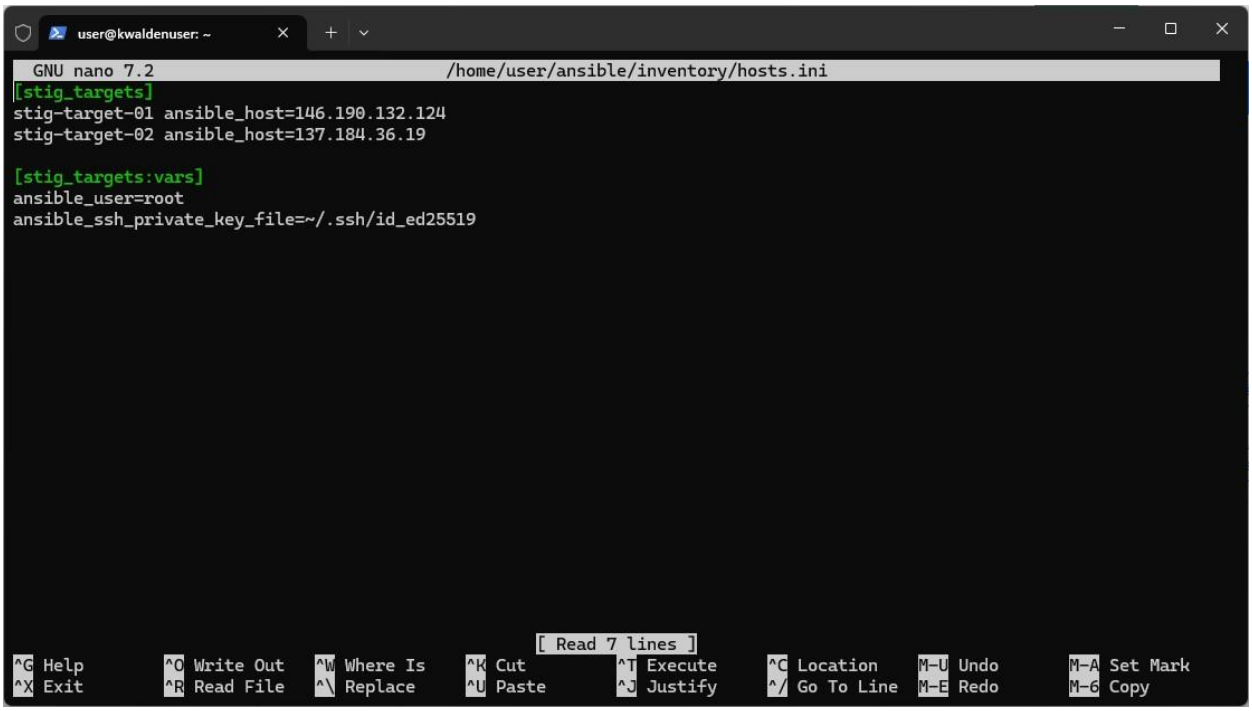
Step 3: Generate and Configure SSH Key Authentication

1. On the Ansible control node, generate an SSH key pair.
2. Ensure the public key is associated with both target droplets in DigitalOcean.
3. Test SSH connectivity from the control node to each target droplet using the root account.

Successful SSH access confirms secure communication between the control node and target systems.

Step 4: Create the Ansible Inventory File

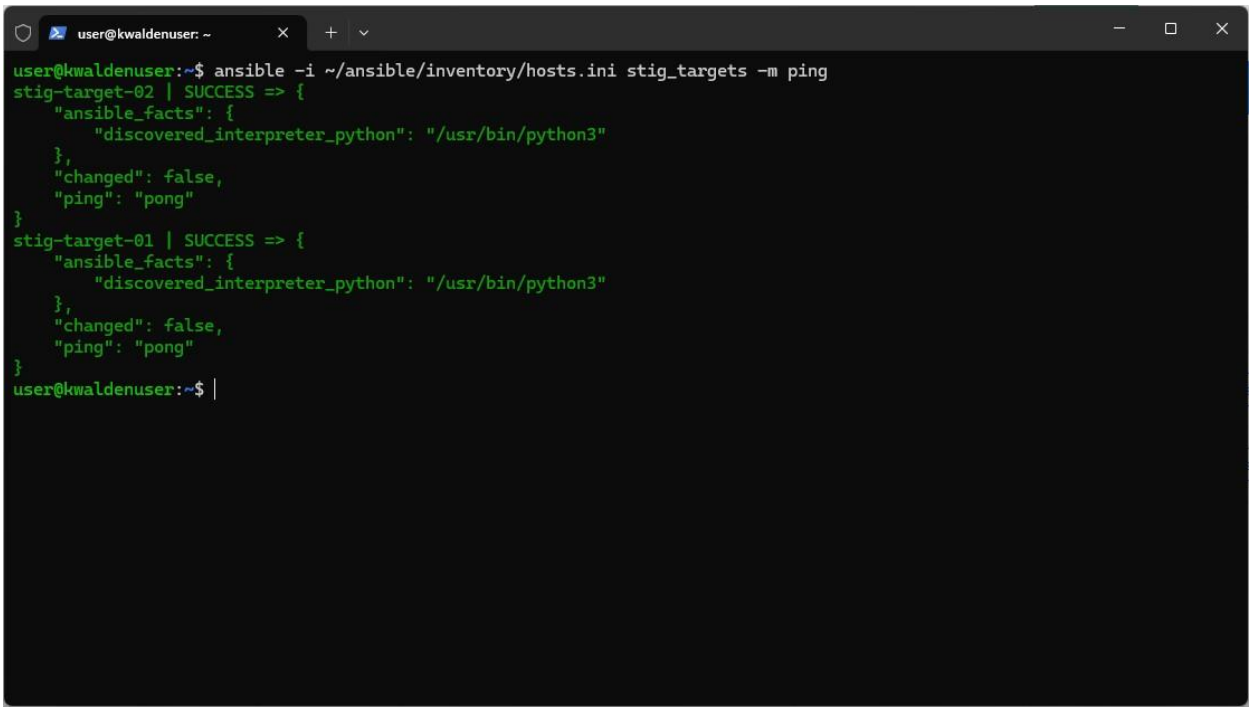
1. On the control node, create an Ansible inventory file.
2. Define both target systems using hostnames mapped to their IP addresses.
3. Specify SSH key usage and remote user configuration.
4. Group both targets under a single host group to support uniform automation.



Step 5: Validate Ansible Connectivity

1. Use the Ansible ping module to test connectivity to both target systems.
2. Confirm both systems return a successful response.
3. Resolve any SSH or inventory parsing issues before proceeding.

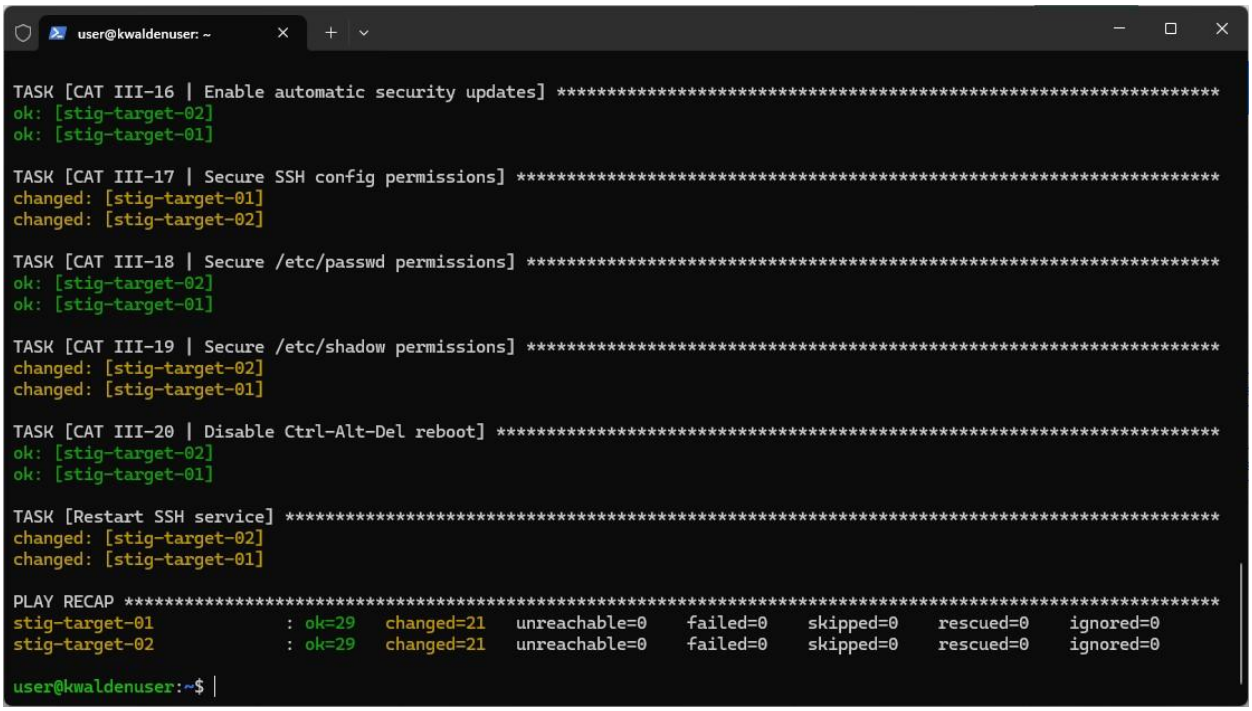
Successful validation confirms Ansible can communicate with both systems.



Step 6: Execute DISA STIG Hardening Playbook

1. Develop an Ansible playbook containing DISA STIG-aligned security controls.
2. Categorize controls by severity: Category I, Category II, and Category III.
3. Execute the playbook against both target systems simultaneously.
4. Monitor playbook execution to ensure no tasks fail.

The playbook applies all security configurations automatically and consistently across both systems.



Step 8: Document Results and Errors

1. Capture screenshots of all successful validation steps.
2. Document any errors encountered during automation or validation.
3. Record error causes and solutions in Appendix A.
4. Update Appendix B with the list of implemented DISA STIGs and risk categories.

FINAL SCRIPT

```
---
- name: Apply DISA STIG-aligned hardening (20 controls)
  hosts: stig_targets
  become: true

  vars:
```

```
  sshd_config: /etc/ssh/sshd_config

tasks:

############################
# CATEGORY I — HIGH (5)
############################

- name: CAT I-01 | Disable SSH root login
  lineinfile:
    path: "{{ sshd_config }}"
    regexp: '^#?PermitRootLogin'
    line: 'PermitRootLogin no'

- name: CAT I-02 | Disable SSH password authentication
  lineinfile:
    path: "{{ sshd_config }}"
    regexp: '^#?PasswordAuthentication'
    line: 'PasswordAuthentication no'

- name: CAT I-03 | Disable empty passwords
  lineinfile:
    path: "{{ sshd_config }}"
    regexp: '^#?PermitEmptyPasswords'
    line: 'PermitEmptyPasswords no'

- name: CAT I-04 | Install and configure UFW firewall
```

```
  apt:
    name: ufw
    state: present

- name: CAT I-04 | Configure firewall defaults
  community.general.ufw:
    direction: incoming
    policy: deny

- name: CAT I-04 | Allow SSH through firewall
  community.general.ufw:
    rule: allow
    name: OpenSSH

- name: CAT I-04 | Enable firewall
  community.general.ufw:
    state: enabled

- name: CAT I-05 | Install and enable auditd
  apt:
    name:
      - auditd
      - audispd-plugins
    state: present

- service:
    name: auditd
```

```
    state: started
    enabled: true

############################
# CATEGORY II — MEDIUM (5)
############################

- name: CAT II-06 | Enforce SSH idle timeout
  lineinfile:
    path: "{{ sshd_config }}"
    regexp: '^#?ClientAliveInterval'
    line: 'ClientAliveInterval 300'

- name: CAT II-07 | Enforce SSH protocol 2
  lineinfile:
    path: "{{ sshd_config }}"
    regexp: '^#?Protocol'
    line: 'Protocol 2'

- name: CAT II-08 | Enforce password aging policies
  lineinfile:
    path: /etc/login.defs
    regexp: '^PASS_MAX_DAYS'
    line: 'PASS_MAX_DAYS 60'

- lineinfile:
    path: /etc/login.defs
```

```
    regexp: '^PASS_MIN_DAYS'
    line: 'PASS_MIN_DAYS 1'

- lineinfile:
    path: /etc/login.defs
    regexp: '^PASS_WARN_AGE'
    line: 'PASS_WARN_AGE 7'

- name: CAT II-09 | Install and enable chrony
  apt:
    name: chrony
    state: present

- service:
    name: chrony
    state: started
    enabled: true

- name: CAT II-10 | Disable unused filesystems
  copy:
    dest: /etc/modprobe.d/stig-filesystems.conf
    content: |
      install cramfs /bin/true
      install squashfs /bin/true

############################
# CATEGORY III — LOW (10)
```

```
############################

- name: CAT III-11 | Disable ICMP redirects
  sysctl:
    name: net.ipv4.conf.all.accept_redirects
    value: '0'
    state: present
    reload: yes

- name: CAT III-12 | Disable source routing
  sysctl:
    name: net.ipv4.conf.all.accept_source_route
    value: '0'
    state: present
    reload: yes

- name: CAT III-13 | Enable TCP SYN cookies
  sysctl:
    name: net.ipv4.tcp_syncookies
    value: '1'
    state: present
    reload: yes

- name: CAT III-14 | Restrict core dumps
  lineinfile:
    path: /etc/security/limits.conf
    line: '* hard core 0'
```

```
- name: CAT III-15 | Set secure default umask
  lineinfile:
    path: /etc/profile
    regexp: '^umask'
    line: 'umask 027'

- name: CAT III-16 | Enable automatic security updates
  apt:
    name: unattended-upgrades
    state: present

- name: CAT III-17 | Secure SSH config permissions
  file:
    path: "{{ sshd_config }}"
    owner: root
    group: root
    mode: '0600'

- name: CAT III-18 | Secure /etc/passwd permissions
  file:
    path: /etc/passwd
    owner: root
    group: root
    mode: '0644'

- name: CAT III-19 | Secure /etc/shadow permissions
```

```
  file:
    path: /etc/shadow
    owner: root
    group: root
    mode: '0640'

- name: CAT III-20 | Disable Ctrl-Alt-Del reboot
  file:
    path: /etc/systemd/system/ctrl-alt-del.target
    state: absent

- name: Restart SSH service
  service:
    name: ssh
    state: restarted
```

LESSONS LEARNED

One of the most significant lessons learned from this project was the value of automation in maintaining consistency and security across multiple systems. Manually configuring security settings on individual servers can easily lead to configuration drift, human error, or overlooked controls. By using Ansible, I was able to enforce the same security baseline across two cloud-hosted systems with a single execution, ensuring identical outcomes. This reinforced the importance of infrastructure-as-code principles in modern system administration and security operations. The project also highlighted how automation improves efficiency by reducing repetitive tasks and allowing administrators to focus on validation and analysis rather than manual configuration. Another key takeaway was understanding how automation tools rely heavily on proper initial setup, particularly SSH authentication and inventory configuration. Small misconfigurations in these foundational areas can prevent automation from functioning entirely. Troubleshooting these issues strengthened my understanding of how Ansible communicates with remote systems. Overall, this project demonstrated that automation is not only a convenience but a critical component of secure and scalable infrastructure management.

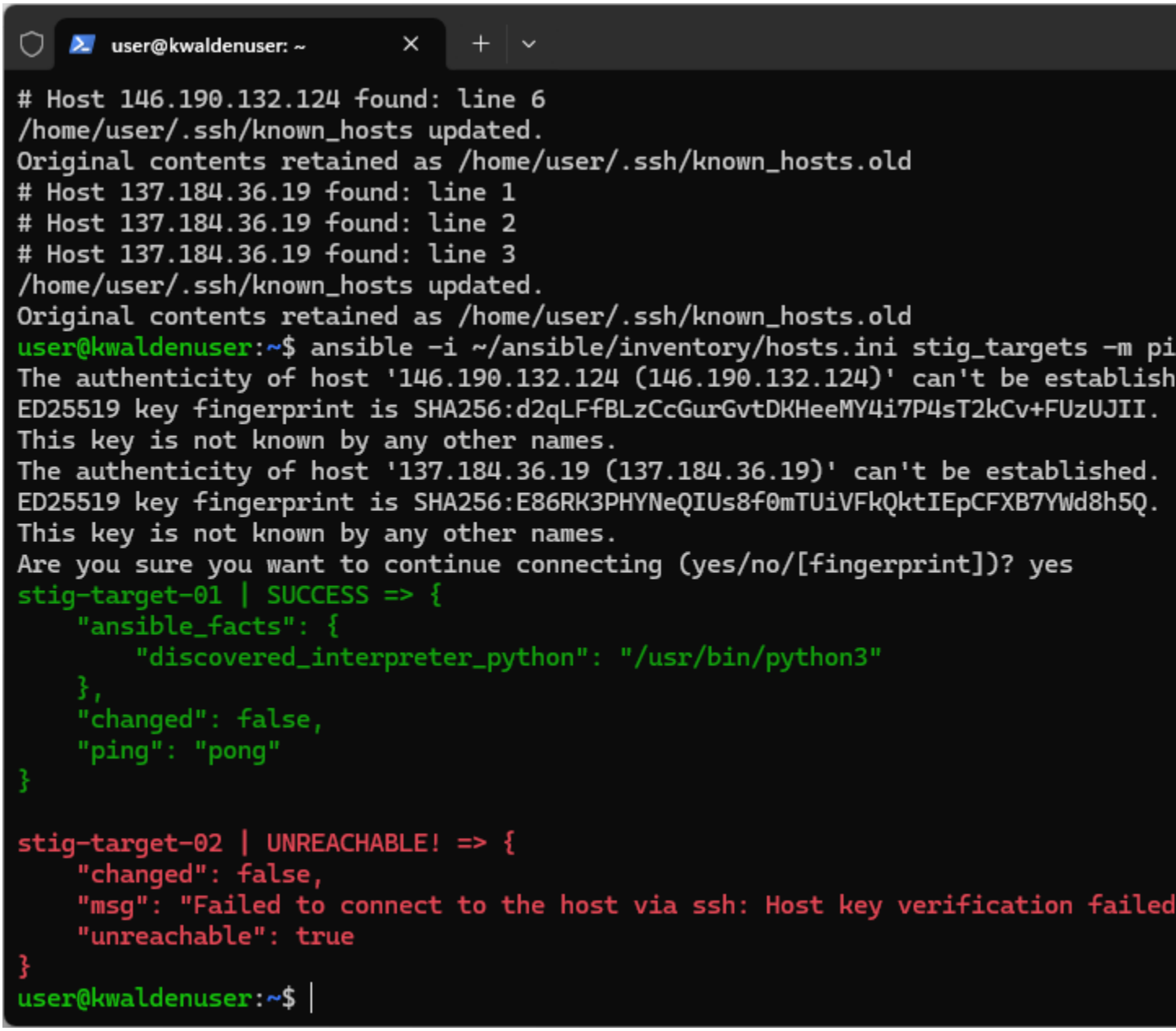
Another important lesson was the practical application of DISA STIGs in a real-world environment. Prior to this project, STIGs were primarily theoretical guidelines; implementing them revealed how they translate into concrete system configurations. Categorizing controls into Category I, II, and III emphasized how risk severity influences prioritization during system hardening. The project also demonstrated that STIG compliance is not about blindly applying settings, but about understanding the security intent behind each control. Mapping STIG identification numbers to actual system changes required careful attention to detail and reinforced the importance of documentation in compliance-driven environments. Additionally, applying STIGs on Ubuntu Linux highlighted the need to adapt compliance frameworks to different operating systems while maintaining their security objectives. This process improved my ability to interpret security requirements rather than treating them as rigid checklists. The experience reinforced that compliance and security are closely linked, but effective implementation requires both technical skill and analytical reasoning. These insights are directly applicable to roles in cybersecurity, system administration, and cloud security engineering.

The project also provided valuable experience in troubleshooting and problem-solving within a cloud-based environment. Several issues were encountered during the setup and validation phases, including SSH authentication failures, incorrect user assumptions, host key verification errors, and Ansible inventory parsing warnings. Resolving these issues required methodical analysis rather than trial-and-error, reinforcing the importance of understanding system behavior at a deeper level. The use of DigitalOcean's console access as a recovery method demonstrated the importance of having fallback access mechanisms when security controls restrict normal administrative access. Additionally, documenting each error and its resolution emphasized the role of clear documentation in long-term system maintenance and knowledge transfer. This process improved my confidence in diagnosing and resolving infrastructure issues under realistic conditions. The project also strengthened my ability to communicate technical challenges and solutions clearly through written documentation. Overall, these lessons reinforced that successful system hardening involves not only applying security controls but also understanding how to recover, validate, and document changes effectively. These skills are essential for working in professional IT and security environments where reliability, accountability, and clarity are critical.

APPENDIX A | DOCUMENT ERRORS & SOLUTIONS

Every time you receive an error while testing your script, take a screenshot of the error, paste it here, explain the error and document how you fixed it. Example:

1 - Error Title: STIG 02 unreachable



Error explanation: The system saw multiple host keys and was detecting a mismatch and refused to trust the host automatically.

Solution: I manually SSH into the droplet with ssh root@137.184.36.19 verified I wanted to continue connecting and logged in successfully.

APPENDIX B | DISA STIGS LIST

List all the STIGs, and the risk category and a brief description of each. See example below.

STIG #	Risk Category	Description
UBTU-22-010040	(Cat 1)	Disable SSH root login
UBTU-22-010060	(Cat 1)	Disable SSH password authentication
UBTU-22-010070	(Cat 1)	Disable empty passwords
UBTU-22-010120	(Cat 1)	Enforce firewall default deny
UBTU-22-030020	(Cat 1)	Enable system auditing
UBTU-22-010090	(Cat 2)	Enforce SSH idle timeout
UBTU-22-010050	(Cat 2)	Enforce SSH protocol 2
UBTU-22-010230	(Cat 2)	Enforce password aging
UBTU-22-020030	(Cat 2)	Enable time synchronization
UBTU-22-010400	(Cat 2)	Disable unused filesystems
UBTU-22-040210	(Cat 3)	Disable ICMP redirects
UBTU-22-040220	(Cat 3)	Disable IP source routing
UBTU-22-040260	(Cat 3)	Enable TCP SYN cookies

UBTU-22-010310	(Cat 3)	Restrict core dumps
UBTU-22-010340	(Cat 3)	Enforce secure default umask
UBTU-22-020080	(Cat 3)	Enable automatic security updates
UBTU-22-010460	(Cat 3)	Secure SSH config permissions
UBTU-22-010450	(Cat 3)	Secure /etc/passwd permissions
UBTU-22-010460	(Cat 3)	Secure /etc/shadow permissions
UBTU-22-020320	(Cat 3)	Disable Ctrl-Alt-Del reboot